



Relatorio Tecnico

Projeto: automation-serverest-victor

Desafio: Desafio Tecnico de QA - Automacao ServeRest

Repositorio: github.com/Victorios990/automation-serverest-victor

Data: 06/07/2026

Documentacao do Produto

Este documento descreve a arquitetura, as boas praticas e o passo a passo de execucao da suite de automacao de testes construida com Cypress para o desafio tecnico de QA da Ambev, cobrindo os fluxos de interface (E2E) e de API do ServeRest.

Controle de Versao

Data da Revisao	Versao	Nome da Revisao	Responsavel
06/07/2026	1.0	Criacao do relatorio tecnico	Victor

Indice

1. Como Executar a Suite
2. Estrutura do Projeto e Pasta support/
3. Boas Praticas Aplicadas
4. Pages e data-testid
5. Estrutura dos Cenarios E2E e API
6. Integracao Continua (CI) e Relatorio Allure

1. Como Executar a Suite

A suite roda contra os ambientes publicos do ServeRest (front.serverest.dev e serverest.dev); nao ha necessidade de subir nada localmente. Pre-requisito: Node.js instalado.

```
npm install # instala as dependencias
npx cypress open # modo interativo (Cypress Test Runner)
npm run cy:run # execucao headless de toda a suite
npm run cy:run:gui # apenas os testes de interface (cypress/e2e/GUI)
npm run cy:run:api # apenas os testes de API (cypress/e2e/API)
npm run lint # checagem de estilo (ESLint + Prettier)
npm run lint:fix # corrige automaticamente o que for possivel
npm run format # formata o codigo com Prettier

# Relatorio Allure (gera allure-results/ durante a execucao):
npm run allure:generate # gera o HTML em allure-report/
npm run allure:open # abre o relatorio no navegador
```

2. Estrutura do Projeto e Pasta support/

```
cypress/
  ■■■ e2e/
  ■ ■■■ GUI/ # cenarios E2E de interface
  ■ ■■■ API/ # cenarios de API
  ■■■ fixtures/
  ■ ■■■ pages/ # mapa de seletores por tela (data-testid)
  ■ ■■■ dados/ # massa de dados estatica (ex.: credenciais invalidas)
  ■■■ support/
  ■ ■■■ actions/ # acoes reaproveitaveis por tela
  ■ ■■■ factories/ # geracao de massa de dados (Faker)
  ■ ■■■ mensagens.js # textos/mensagens esperadas, centralizados
  ■ ■■■ imports.js # barrel de imports usados pelos specs
  ■ ■■■ commands.js # comandos customizados (setup/teardown via API)
  ■ ■■■ e2e.js # configuracao global (afterEach, uncaught:exception)
  ■■■ ...
```

Detalhamento dos arquivos de support/

- **commands.js**: comandos customizados do Cypress usados para setup/teardown via API (criarUsuarioViaApi, autenticarViaApi, buscarUsuarioPorEmailViaApi, excluirUsuarioViaApi), alem do par **registrarUsuarioParaLimpeza / limparUsuariosRegistrados**, que centraliza a limpeza de dados de todos os specs em um unico lugar.
- **e2e.js**: ponto de entrada de configuracao global. Registra um unico *afterEach* global que chama **cy.limparUsuariosRegistrados()**, eliminando a duplicacao de logica de teardown em cada spec. Tambem trata excecoes nao capturadas da aplicacao (uncaught:exception) para nao quebrar o teste por erros do proprio front alheios ao cenario testado.
- **mensagens.js**: centraliza todos os textos esperados (mensagens de erro/sucesso da API e da interface), evitando strings magicas espalhadas pelos specs e facilitando a manutencao caso o texto mude em um unico lugar.
- **imports.js**: barrel file que reexporta faker, mensagens, as factories e as actions, permitindo que cada spec faca um unico import consolidado em vez de multiplas linhas de import.
- **factories/**: geradores de massa de dados dinamica (UsuarioFactory, ProdutoFactory) baseados em @faker-js/faker, com e-mail/nome unicos por execucao (timestamp + sufixo aleatorio), evitando colisao de dados no ambiente publico e compartilhado do ServeRest.
- **actions/**: encapsulam as interacoes de cada tela (preencher formulario, submeter, logout etc.) como funcoes nomeadas, evitando duplicar sequencias de cy.get().type() em cada spec.

3. Boas Praticas Aplicadas

- **Selecao por data-testid:** sempre que disponivel, os seletores usam os atributos data-testid expostos pelo proprio ServeRest — mais estaveis que classes/ids de CSS e resistentes a mudancas de estilo.
- **Seletores centralizados:** nenhum seletor fica hardcoded nos specs; todos vivem em fixtures/pages/*.json, entao qualquer mudanca de layout se resolve em um unico lugar.
- **Independencia entre testes:** cada cenario cria sua propria massa de dados (via API, com Faker) e faz sua propria limpeza no afterEach — nenhum spec depende da ordem de execucao ou do estado deixado por outro.
- **Dados dinamicos:** uso de @faker-js/faker para gerar e-mails/nomes unicos por execucao, evitando testes frageis por colisao de dados no ambiente publico do ServeRest.
- **Separação GUI x API:** testes de API usam cy.request diretamente, sem depender da camada de interface, tornando-os mais rapidos e menos suscetiveis a instabilidade de renderizacao.
- **Sem mocks no E2E:** deliberadamente nao se usa cy.intercept para simular respostas — todos os cenarios validam o comportamento real da aplicacao publica, mantendo o carater de teste de ponta a ponta genuino exigido pelo desafio.
- **Padronizacao de codigo:** ESLint (eslint-plugin-cypress) + Prettier garantem estilo consistente; ambos rodam tambem como um job dedicado no CI.
- **Relatorio de execucao:** integracao com Allure Report para visualizacao detalhada de cada execucao (historico, passos, evidencias de falha).
- **Mensagens centralizadas:** nenhuma string de validacao/erro/sucesso e duplicada entre specs — todas vem de support/mensagens.js.
- **Cobertura completa do contrato de API:** a suite de API foi expandida para cobrir 100% das rotas documentadas no Swagger oficial do ServeRest (login, usuarios, produtos, carrinhos - incluindo listagem com filtros, edicao via PUT, exclusao explicita e as duas variantes de encerramento de carrinho, cancelar-compra e concluir-compra, que tem efeitos diferentes sobre o estoque e sao validadas separadamente).
- **Instalacao fora de pastas sincronizadas na nuvem:** recomenda-se rodar npm install fora de pastas com iCloud Drive/Dropbox/OneDrive ativos, pois a sincronizacao em tempo real pode interferir na escrita de arquivos durante a instalacao e corromper dependencias do Cypress.

4. Pages e data-testid

Cada tela da aplicacao possui um arquivo JSON correspondente em `cypress/fixtures/pages/`, mapeando um nome semantico para o seletor real (prioritariamente `data-testid`). Os specs e as actions nunca referenciam o seletor diretamente — sempre atraves desse mapa.

4.1 loginPage.json

```
"campoEmail": "[data-testid=\"email\"]"
"campoSenha": "[data-testid=\"senha\"]"
"botaoEntrar": "[data-testid=\"entrar\"]"
"alertaErro": ".alert"
```

4.2 cadastroPage.json

```
"nomeInput": "[data-testid=\"nome\"]"
"emailInput": "[data-testid=\"email\"]"
"passwordInput": "[data-testid=\"password\"]"
"administradorCheckbox": "[data-testid=\"checkbox\"]"
"botaoCadastrar": "[data-testid=\"cadastrar\"]"
"linkEntrar": "[data-testid=\"entrar\"]"
"alertaErro": ".alert"
```

4.3 homePage.json

```
"navHome": "[data-testid=\"home\"]"
"navListaDeCompras": "[data-testid=\"lista-de-compras\"]"
"navCarrinho": "[data-testid=\"carrinho\"]"
"botaoLogout": "[data-testid=\"logout\"]"
"campoPesquisa": "[data-testid=\"pesquisar\"]"
"botaoPesquisar": "[data-testid=\"botaoPesquisar\"]"
"cardProduto": ".card.col-3"
"tituloProduto": ".card-title"
"botaoAdicionarNaLista": "[data-testid=\"adicionarNaLista\"]"
```

4.4 minhaListaDeProdutosPage.json

Mapeia os seletores da tela de lista de compras (itens adicionados, campo de quantidade e subtotal calculado), reaproveitados pelos cenarios CT08 e CT09.

```
"botaoAdicionarCarrinho": "[data-testid=\"adicionar carrinho\"]"
"botaoLimparLista": "[data-testid=\"limparLista\"]"
"nomeProduto": "[data-testid=\"shopping-cart-product-name\"]"
"quantidadeProduto": "[data-testid=\"shopping-cart-product-quantity\"]"
"botaoAumentarQuantidade": "[data-testid=\"product-increase-quantity\"]"
"botaoDiminuirQuantidade": "[data-testid=\"product-decrease-quantity\"]"
```

5. Estrutura dos Cenarios E2E e API

Cada spec segue o padrao describe/it do Mocha/Cypress, com nomenclatura de casos no formato CTxx - descricao do cenario. Especificamente:

- **GUI (cypress/e2e/GUI):** usa as actions (CadastroActions, LoginActions, HomeActions) para executar as interacoes de tela, e os seletores vem sempre de fixtures/pages/*.json. A massa de dados usada no login/cadastro e criada via API (mais rapido e estavel que criar pela propria interface) e registrada para limpeza com cy.registrarUsuarioParaLimpeza.
- **API (cypress/e2e/API):** usa cy.request diretamente contra Cypress.env('apiUrl'). Helpers locais como criarAdminAutenticado()/criarUsuarioComumAutenticado() encapsulam o pre-requisito comum de varios cenarios (usuario + token). O teardown de carrinho segue a ordem exigida pela API (cancelar carrinho antes de excluir produto e usuario).
- **Teardown global:** um unico afterEach em cypress/support/e2e.js chama cy.limparUsuariosRegistrados(), que percorre a lista de usuarios registrados durante o teste (por id ou por email/senha) e remove cada um via API, garantindo que nenhum residuo de teste fique no ambiente publico compartilhado.
- **Sem numeracao de ordem de execucao:** os nomes dos arquivos de spec nao seguem numeracao sequencial de proposito — cada teste e independente, podendo rodar sozinho, fora de ordem ou em paralelo.

6. Integracao Continua (CI) e Relatorio Allure

O workflow `.github/workflows/cypress.yml` roda no GitHub Actions a cada push/PR na branch main, com os seguintes jobs:

- **lint**: ESLint + verificacao de formatacao (Prettier).
- **e2e-gui**: executa a suite de testes de interface.
- **api**: executa a suite de testes de API.
- **allure-report**: junta os resultados dos dois jobs acima e publica o relatorio HTML do Allure como artefato do workflow.

Screenshots de falhas e o relatorio Allure ficam disponiveis para download na aba Actions do GitHub, no run correspondente — o que permite investigar qualquer falha sem precisar reexecutar a suite localmente.